

Hoja de trabajo 5

Orquestación de agentes para Parachute S.A.

1. ¿Qué arquitectura/arquitecturas resuelven mejor este problema? ¿Por qué?

Para las capacidades actuales de Parachute S.A., la arquitectura centralizada es la alternativa más sencilla de organizar: un manager coordina FAQs, clima y reservas. La secuencia de consulta y calendarización es corta y resulta fácil seguir quién delega cada operación. Su desventaja aparece si se acumulan muchas capacidades e instrucciones en el mismo manager.

Considerando que la empresa seguirá aumentando sus requerimientos, preferimos la arquitectura jerárquica. El supervisor separa información y operaciones; el submanager operativo puede incorporar disponibilidad de instructores, equipo o pagos sin obligar al supervisor a conocer cada detalle. Esta separación facilita mantener responsabilidades por dominio. El costo es una capa adicional de coordinación, que puede aumentar llamadas y latencia. No se midieron esas variables, por lo que no afirmamos una ventaja cuantitativa.

La descentralizada resulta útil cuando la conversación cambia de especialidad: un agente transfiere el control a otro y el receptor continúa con el historial. En nuestra implementación, FAQs transfiere a Clima y Clima puede transferir a Reservas; Reservas puede regresar a FAQs o Clima. El agente activo responde directamente. Su dificultad es mantener las rutas de transferencia, evitar ciclos y conservar la intención del usuario al cambiar de especialista.

La ausencia de un manager no obliga a duplicar las reglas meteorológicas. Las tres arquitecturas reutilizan el mismo evaluador y el servicio de reservas consulta el clima antes de escribir en SQLite. Esa garantía vive en código, no solamente en instrucciones de agentes. Por tanto, la descentralizada también puede cumplir la restricción de comprobar el clima antes de calendarizar.

Elegiríamos centralizada para el alcance pequeño actual y jerárquica si el crecimiento esperado se organiza por dominios. La elección no depende solo del número de agentes: también importan el mantenimiento, la trazabilidad, el costo y los resultados de pruebas con solicitudes reales.

Esta respuesta integra la recomendación inicial del integrante 2 y la actualiza con la implementación descentralizada. Corrige la suposición del borrador inicial de que los handoffs obligan a repetir reglas de negocio en prompts.

2. ¿Considera que es necesario utilizar un sistema multiagente en este caso? ¿Por qué?

No es estrictamente necesario para los requisitos actuales. Un solo agente con herramientas de búsqueda de FAQs, consulta del clima y registro de citas, acompañado de un flujo determinista de validación, puede resolver el problema. El lenguaje natural ayuda a identificar la intención y reunir datos, pero los umbrales meteorológicos, la validación de fechas y la prevención de duplicados no requieren varios modelos razonando.

En este laboratorio sí usamos tres sistemas multiagente porque el objetivo es comparar arquitecturas. La división permite observar responsabilidades, delegación y transferencia de control. En un producto podría ser útil si las capacidades crecen y requieren instrucciones, herramientas o permisos distintos por dominio. Ese beneficio debe compensar la mayor complejidad de coordinación, los posibles ciclos y el costo de más llamadas al modelo.

Las abstracciones compartidas ofrecen valor incluso sin MAS: el cliente Open-Meteo adapta datos; el evaluador aplica umbrales; la búsqueda recupera FAQs con sus fuentes; y el servicio de reservas verifica condiciones y persiste citas. Se puede cambiar la orquestación sin reescribir esas integraciones.

Nuestra recomendación práctica sería comenzar con un agente y herramientas si solo se necesita el alcance actual. Introduciríamos múltiples agentes cuando la separación por dominio aporte una mejora verificable. El crecimiento futuro justifica diseñar módulos extensibles, pero no prueba por sí solo que un MAS sea indispensable.

Evidencia y alcance

Las pruebas automatizadas comprueban reglas climáticas, persistencia SQLite, prevención de duplicados y continuidad de los handoffs con el Runner del SDK y un modelo simulado. Se consultó Open-Meteo real sin crear una reserva. No se midieron costos, latencias ni calidad de respuestas de un LLM real; tampoco se verificó PostgreSQL real en este entorno. Esas limitaciones impiden afirmar que una arquitectura sea experimentalmente superior a las demás.

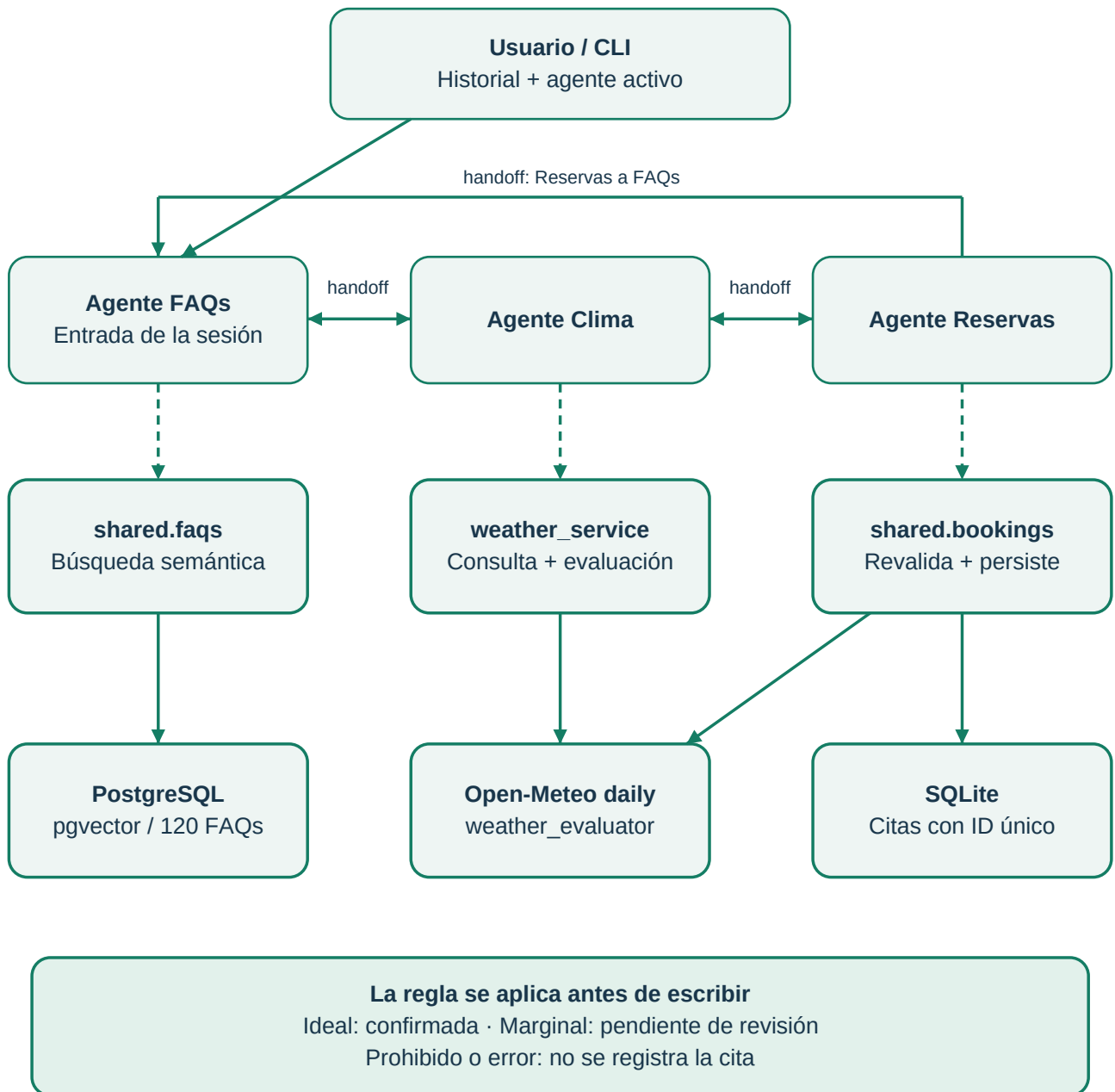
Las condiciones marginales se guardan pendientes de revisión, las prohibidas se rechazan y las ideales permiten confirmar. Esta política es compartida por las tres arquitecturas; no es una propiedad intrínseca de MAS.

Referencias

- Enunciado de la Hoja de trabajo 5 y código del equipo en este repositorio.
- OpenAI, inicio rápido del Agents SDK: <https://developers.openai.com/api/docs/quickstart>
- Open-Meteo, Forecast API: <https://open-meteo.com/en/docs>

Arquitectura descentralizada

Parte 3: especialistas pares y servicios compartidos



Las flechas entre agentes son handoffs. Las flechas punteadas son llamadas a herramientas. La CLI continúa con el último agente y el historial; no hay un manager que reciba todos los resultados. El Runner limita cada solicitud a 12 turnos para acotar ciclos.